SOFTWARE REQUIREMENTS SPECIFICATION

Cyber Security Awareness Game

IEEE-Style SRS Document

Document Control

Item	Details
Project Title	Cyber Security Awareness Game
Document Type	Software Requirements Specification (SRS)
Reference Style	IEEE 830 / IEEE-style requirements structure
Application	Java desktop application
GUI Framework	Java Swing
Database	SQLite
Database Access	JDBC
Primary Users	Students / general users learning cybersecurity awareness

Table of Contents

1. Introduction
2. Overall Description
3. Specific Requirements
4. External Interface Requirements
5. System Design and Architecture
6. UML and Data Flow Diagrams
7. Database Design
8. Testing Requirements
9. Constraints, Assumptions and Risks

10. Future Enhancements

11. Conclusion

1. Introduction

1.1 Purpose

The purpose of the Cyber Security Awareness Game is to provide an interactive desktop application that teaches users basic cybersecurity practices through short explanations and multiple-choice quizzes. The system combines learning content, five cybersecurity modules, quiz scoring, a graphical user interface, and SQLite-based user storage.

1.2 Scope

The application allows a user to enter a defender name, select Hinglish or English as a language option, open a cybersecurity module, read an explanation, answer five multiple-choice questions, receive immediate feedback, earn points, and view a final score. The implemented code currently stores the username, initial score, and selected language in a SQLite database.

1.3 Intended Audience

This SRS is intended for students, project developers, software engineering faculty, testers, and users who need a clear description of the system requirements and design.

1.4 Definitions and Acronyms

Term	Meaning
SRS	Software Requirements Specification
GUI	Graphical User Interface
JDBC	Java Database Connectivity
SQLite	Lightweight embedded relational database
2FA	Two-Factor Authentication
OTP	One-Time Password
DB	Database

2. Overall Description

2.1 Product Perspective

The system is a standalone Java Swing desktop application. It uses CardLayout for navigation between login and dashboard screens, separate JFrame windows for modules and final results, and JDBC for communication with an SQLite database.

2.2 Product Functions

• User login/setup using a defender name.

• Language selection between Hinglish and English.

• Dashboard containing five cybersecurity modules and a final challenge/result option.

• Learning screen containing topic explanations and real-world examples.

• Five MCQ questions per module.

• Immediate correct/incorrect feedback.

• 20 points for each correct answer.

• Module score out of 100 and overall score out of 500.

• SQLite storage of username, score field and selected language.

2.3 User Classes

User Class	Description
Learner / Defender	Uses modules, reads explanations, answers quizzes and views results.
Developer / Administrator	Maintains source code, quiz content, database structure and application configuration.

2.4 Operating Environment

The application requires a Java Runtime Environment/JDK capable of running Swing applications, the SQLite JDBC driver, and a desktop operating system such as Windows, Linux or macOS.

2.5 Design and Implementation Constraints

• Desktop-only Java Swing implementation.

• SQLite database is local to the application.

• The current content is stored directly in Java arrays.

• The current GUI language selection changes the displayed language label/title but does not provide a secondEnglish content set.

• The database connection errors are currently suppressed by the connectDB() catch block.

• Final score is maintained in application memory; the current code does not issue a database UPDATE for earnedpoints.

3. Specific Requirements

3.1 Functional Requirements

ID	Requirement	Description
FR-01	User Identification	The system shall allow the user to enter a Defender Name before starting.
FR-02	Input Validation	The system shall prevent the user from continuing when the Defender Name is empty.
FR-03	Language Selection	The system shall provide Hinglish and English options.
FR-04	User Storage	The system shall insert a new user record into the SQLite users table when the game starts.
FR-05	Dashboard	The system shall display five learning modules and a final result option.
FR-06	Module Access	The system shall open the selected module in a separate window.
FR-07	Learning Content	The system shall display a topic explanation and real-world cybersecurity example.
FR-08	Quiz	Each module shall provide five multiple-choice questions.
FR-09	Answer Validation	The system shall compare the selected answer with the predefined correct answer.
FR-10	Scoring	The system shall award 20 points for each correct answer.
FR-11	Feedback	The system shall display immediate correct or incorrect feedback.
FR-12	Module Completion	After five questions, the system shall show the module score out of 100 and close the module window.
FR-13	Final Score	The system shall provide an option to view the accumulated score out of 500.
FR-14	Database	The system shall create the users table if it does not already exist.

3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Usability	The GUI should be simple and understandable for beginner cybersecurity learners.
NFR-02	Performance	Screen navigation and quiz response should occur without noticeable delay on a normal desktop.
NFR-03	Reliability	The application should handle empty input and unselected quiz options without crashing.
NFR-04	Maintainability	Module content and correct answers should be organized so they can be updated by the developer.
NFR-05	Portability	The Java application should be usable on operating systems with a compatible JDK.

NFR-06	Security Awareness	The content should promote safe practices such as avoiding phishing, using strong passwords, avoiding unknown USB devices and protecting credentials.
NFR-07	Data Integrity	User records stored in SQLite should follow the defined table structure.

4. External Interface Requirements

4.1 User Interface

• Login screen: Defender Name field, language selection box and Launch Mission button.

• Dashboard: six buttons representing five modules plus final challenge/result.

• Module explanation screen: topic title, explanation text, real-world example and quiz start button.

• Quiz screen: question label, three radio-button choices and Submit Answer button.

• Result screen: mission completion message, total score and Exit Dashboard button.

4.2 Software Interfaces

Java Swing provides GUI components such as JFrame, JPanel, JLabel, JTextField, JComboBox, JButton, JRadioButton, ButtonGroup, JTextArea and JScrollPane. JDBC provides database connectivity to SQLite through the org.sqlite.JDBC driver.

4.3 Database Interface

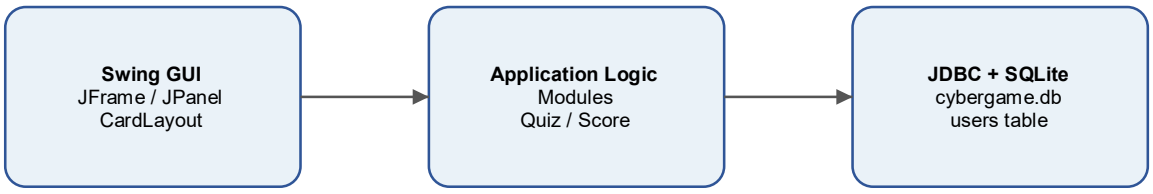
The application creates or opens the local file cybergame.db and creates the users table using SQL when required.

4.4 Hardware Interface

No special hardware is required. The system runs on a standard computer with keyboard, mouse and display.

5. System Design and Architecture

5.1 High-Level Architecture



The architecture can be viewed as three main layers: Swing presentation, application/quiz logic, and JDBC/SQLite persistence. The current implementation places these responsibilities within the CyberSecurityGameApp class and its inner classes.

5.2 Module Structure

Module	Main Content	Questions	Maximum Score
1. Phishing & Email Safety	Phishing, fake emails, spear phishing	5	100
2. Password Vault & Strength	Strong passwords, managers, 2FA	5	100
3. Malware & Ransomware	Malware, ransomware, antivirus, safe downloads	5	100
4. Social Engineering	OTP scams, tailgating, vishing, manipulation	5	100
5. Safe Browsing & Wi-Fi	HTTPS, public Wi-Fi, incognito, updates	5	100

7.5 Database Observation

The current source code inserts the initial score as 0 but does not update the database when quiz points are earned. Therefore, persistent final-score storage is a recommended enhancement rather than a currently implemented requirement.

8. Detailed Module Requirements

Module 1: Phishing & Email Safety

Explains phishing, urgent fake bank messages, spear phishing and suspicious links. Five MCQs test recognition and response to phishing attempts.

Module 2: Password Vault & Strength

Explains strong passwords, password length, unique passwords, password managers and 2FA. Five MCQs test password security awareness.

Module 3: Malware & Ransomware

Explains malware, ransomware, antivirus, cracked software and unknown USB devices. Five MCQs test safe malware-prevention practices.

Module 4: Social Engineering

Explains manipulation through fear, trust and greed, including OTP scams, tailgating and vishing. Five MCQs test safe responses.

Module 5: Safe Browsing & Wi-Fi

Explains HTTPS, public Wi-Fi risks, private browsing, pop-ups and browser updates. Five MCQs test safe browsing behavior.

8.1 Quiz Scoring Logic

Each module contains five questions. A correct response adds 20 points to both the module score and totalScore. An incorrect response adds zero points. Therefore, the maximum module score is 100 and the theoretical maximum total score across five modules is 500.

8.2 Feedback Logic

The application displays a dialog after every submitted answer. Correct answers show a positive feedback message with +20 points, while incorrect answers show a negative feedback message.

9. Testing Requirements

9.1 Test Cases

TC-ID	Test Case	Input/Action	Expected Result
TC-01	Empty username	Click Launch Mission without name	Validation message appears and dashboard is not opened.
TC-02	Valid login	Enter name and click Launch Mission	User record is inserted and dashboard opens.
TC-03	Language selection	Choose Hinglish/English	Selected value is stored and used in module title.
TC-04	Open module	Click any module button	Corresponding ModuleFrame opens.
TC-05	Explanation	Open module	Explanation and real-world example are visible.
TC-06	No answer	Click Submit without selecting	Prompt asks user to select an option.
TC-07	Correct answer	Select correct option	+20 points and correct feedback appear.
TC-08	Wrong answer	Select wrong option	Incorrect feedback appears; no points added.
TC-09	Module completion	Submit fifth question	Module completion score out of 100 appears and window closes.
TC-10	Final result	Click final challenge/result	Accumulated total score out of 500 is displayed.
TC-11	Database creation	Run application with no database/table	cybergame.db and users table are created.
TC-12	Database failure	Make SQLite connection unavailable	Application should ideally report the database problem; current code silently returns null.

9.2 Acceptance Criteria

• The user can start the application and enter a valid name.

• The dashboard provides access to all five modules.

• Each module presents an explanation followed by five questions.

• Correct answers produce 20 points.

• The module score reaches a maximum of 100.

• The final result can display the accumulated score up to 500.

• The SQLite users table is created automatically when needed.

10. Constraints, Assumptions and Risks

10.1 Constraints

• Requires Java and the SQLite JDBC driver.

• Content is embedded in Java source code.

• The application is currently a desktop application.

• Only one simple users table is defined.

• Language-specific content is not fully separated in the current implementation.

10.2 Assumptions

• The user has basic computer knowledge.

• The SQLite JDBC driver is available in the project classpath.

• The local database file is writable.

• The user understands that the game is an awareness/educational tool, not a cybersecurity protection product.

10.3 Risks

• Repeatedly opening the same module can increase totalScore because replay protection is not implemented.

• Database errors are hidden, which may make troubleshooting difficult.

• Scores are not persisted after each answer in the current implementation.

• Hard-coded questions require source-code changes for content updates.

11. Future Enhancements

• Implement complete English and Hinglish content sets.

• Persist the updated score in SQLite after each correct answer or module completion.

• Prevent duplicate scoring when a module is replayed.

• Add a proper final certificate with username, score and completion date.

• Add leaderboard functionality.

• Add user login/history and progress tracking.

• Move quiz questions into database tables so content can be updated without recompiling.

• Add administrator functionality for managing questions.

• Add timer-based challenges and difficulty levels.

• Add more cybersecurity topics such as mobile security, social media safety and secure coding.

• Add secure password hashing if actual account authentication is introduced.

• Improve database error reporting and logging.

12. Conclusion

The Cyber Security Awareness Game is a Java Swing desktop application designed to improve basic cybersecurity awareness through interactive learning and quizzes. Its five modules cover phishing, password security, malware/ransomware, social engineering, and safe browsing/Wi-Fi. The current implementation demonstrates GUI navigation, MCQ validation, scoring, JDBC connectivity and SQLite user storage. This SRS defines the implemented behavior and identifies practical enhancements required for a more complete production-quality system.

Appendix A — Implementation Mapping

Code Element	SRS Responsibility
CyberSecurityGameApp	Main application controller and dashboard/login management
createLoginPanel()	User identification and language selection
connectDB()	SQLite/JDBC connection and users table creation
createDashboardPanel()	Module and final-result navigation
ModuleFrame	Learning content, quiz UI and module scoring
createExplainPanel()	Module explanation and real-world example
createQuizPanel()	MCQ interface and answer submission
loadQ()	Loads questions and handles module completion
FinalResultFrame	Displays accumulated final score
moduleContent	Stores five modules, explanations and questions

correctAnswers	Stores predefined correct options
totalScore	Stores accumulated score during current application run

Appendix B — Current Implementation Notes

The diagrams and requirements in this document describe the code supplied for the project. Features that are recommended but not currently implemented—such as persistent score updates, full language translation, replay protection and certificate generation—are listed as future enhancements rather than represented as existing capabilities.